

DOMAIN 2 — DATABASE OBJECT MANAGEMENT

INTRODUCTION TO DATABASE OBJECT MANAGEMENT

Database Object Management focuses on creating, modifying, organizing, and managing objects inside a database. These objects include tables, views, stored procedures, and indexes.

Database administrators and developers use SQL (Structured Query Language) commands to interact with database objects. Understanding these commands is essential because they are commonly used in both the Certiport certification exam and real-world database systems.

The major goals of database object management are:

- organize data properly
- improve database performance
- maintain database structure
- simplify data access
- support efficient data retrieval

TABLES

Tables are the primary objects used to store data inside a database. A table contains:

- columns (fields)
- rows (records)

Each table focuses on a single category of information.

Examples:

Employees
Students
Products
Orders

CREATE TABLE

The CREATE TABLE statement is used to create a new table inside a database. When creating a table, developers define:

- column names
- data types

- constraints
- primary keys

BASIC SYNTAX

```
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    Age INT
);
```

EXPLANATION

Part	Purpose
CREATE TABLE	Creates a new table
StudentID	Column name
INT	Data type
PRIMARY KEY	Unique identifier
VARCHAR(50)	Stores text

IMPORTANT RULES WHEN CREATING TABLES

- ✓ Column names should be meaningful
- ✓ Avoid spaces in field names
- ✓ Define appropriate data types
- ✓ Use primary keys whenever possible
- ✓ Required fields should use NOT NULL

NULL AND NOT NULL - Columns may either allow or reject empty values.

NULL - A NULL value means:

- no value
- missing value
- unknown value

NULL is NOT:

- zero
- blank text
- NOT NULL

NOT NULL means a field is required and cannot be empty.

EXAMPLE

```
CREATE TABLE Employees (  
    EmployeeID INT PRIMARY KEY,  
    FullName VARCHAR(100) NOT NULL  
);
```

In this example:

FullName must contain a value.

ALTER TABLE

The ALTER TABLE command modifies an existing table.

Developers use ALTER TABLE to:

- add columns
- modify columns
- remove columns
- change constraints
- ADDING A COLUMN
- ALTER TABLE Students
- ADD Email VARCHAR(100); - This adds an Email column to the Students table.

MODIFYING A COLUMN

ALTER TABLE Students

```
ALTER COLUMN Email VARCHAR(200);
```

This changes the size of the Email field.

DROP TABLE - The DROP TABLE command permanently removes a table from the database.

EXAMPLE

```
DROP TABLE Students;
```

IMPORTANT WARNING

Dropping a table:

- deletes structure
- deletes records
- cannot easily be undone

Developers should use this carefully.

VIEWS - A view is a saved SQL query inside a database. Views do not usually store data physically. Instead, they display data from underlying tables.

Views help:

- simplify complex queries
- improve security
- organize frequently used queries

CREATE VIEW

```
CREATE VIEW StudentView AS
SELECT FirstName, LastName
FROM Students;
```

This creates a virtual table called StudentView.

ADVANTAGES OF VIEWS

- ✓ Simplify reporting
- ✓ Hide unnecessary columns
- ✓ Improve security
- ✓ Reuse complex queries

ALTER VIEW - The ALTER VIEW statement changes an existing view.

EXAMPLE

```
ALTER VIEW StudentView AS
SELECT FirstName, LastName, Age
FROM Students;
```

This updates the view to include Age.

DROP VIEW - The DROP VIEW statement removes a view.

EXAMPLE

```
DROP VIEW StudentView;
```

STORED PROCEDURES

A stored procedure is a saved SQL program that performs a task. Stored procedures help:

- automate processes
- reuse SQL code
- improve security
- reduce repetitive coding

INPUT PARAMETERS - Input parameters allow users to pass values into a procedure.

EXAMPLE

```
CREATE PROCEDURE GetStudent  
@StudentID INT  
AS  
SELECT * FROM Students  
WHERE StudentID = @StudentID;
```

The procedure accepts StudentID as input.

OUTPUT PARAMETERS - Output parameters return values back to applications or users.

RETURN VALUES - Stored procedures may return:

- result sets
- calculations
- status codes

ADVANTAGES OF STORED PROCEDURES

- ✓ Faster execution
- ✓ Better security

- ✓ Reusable logic
- ✓ Easier maintenance

INDEXES

Indexes improve query performance by helping databases locate data faster. Indexes work similarly to a book index:

- instead of scanning every page,
- the system quickly finds the needed information.

CLUSTERED INDEX - A clustered index organizes records physically based on the primary key. Only one clustered index is usually allowed per table.

BENEFITS OF CLUSTERED INDEXES

- ✓ Faster searches using primary key
- ✓ Improved sorting performance
- ✓ Efficient retrieval of ordered data

NONCLUSTERED INDEX - A nonclustered index creates a separate searchable structure. Unlike clustered indexes:

- multiple nonclustered indexes can exist in one table.

ADVANTAGES OF NONCLUSTERED INDEXES

- ✓ Faster searching on non-primary fields
- ✓ Improved filtering performance
- ✓ Better query optimization

DISADVANTAGES OF INDEXES - Although indexes improve performance, they:

- consume storage space
- may slow INSERT/UPDATE operations
- require maintenance

Indexes should only be created when necessary.

CLUSTERED VS NONCLUSTERED INDEX

Clustered Index

Uses physical table order
Usually primary key
One per table

Nonclustered Index

Separate structure
Can use other fields
Multiple allowed

Faster ordered retrieval

Faster searching/filtering

COMMON DOMAIN 2 EXAM QUESTIONS

What command creates a table?

✓ CREATE TABLE

What command modifies a table?

✓ ALTER TABLE

What command removes a table?

✓ DROP TABLE

What is a saved query?

✓ View

What improves query performance?

✓ Indexes

Which index uses physical ordering?

✓ Clustered Index

Which constraint prevents empty values?

✓ NOT NULL

IMPORTANT SQL COMMANDS TO MEMORIZE

Command	Purpose
CREATE TABLE	Create table
ALTER TABLE	Modify table
DROP TABLE	Delete table
CREATE VIEW	Create view
DROP VIEW	Remove view
CREATE PROCEDURE	Create stored procedure